Namo Presentation Examples with Comparisons

Date: 20260615

Companion document to the Namo Roadmap. Each example shows one comparison tool (the strongest competitor for that discipline), then three stages of Namo — 1.x (explicit), 2.x (bare names), 3.x (DSL) — so the audience watches the ceremony disappear.

The narrative: "This is what you're writing now. This is 1.0. This is 2.0. This is 3.0." Four stages per discipline. Each step removes noise. By 3.x, the specification reads like English.

Comparison tools: Pandas, Polars, xarray, R/dplyr, Julia/DataFrames.jl — each chosen as the most credible competitor for that audience.

Import/require lines are omitted throughout. The audience is looking at the logic, not the boilerplate.

Finance / trading

Compose OHLCV price data with quarterly fundamentals. Define indicators, score, decide.

Pandas

```
ohlcv = ohlcv.sort_values('date')
fundamentals = fundamentals.sort_values('quarter_end')
analysis = pd.merge_asof(
    ohlcv, fundamentals,
    left_on='date', right_on='quarter_end',
    by='symbol', direction='backward'
)

analysis['sma_20'] = analysis.groupby('symbol')['close'].transform(
    lambda x: x.rolling(20).mean()
)
analysis['sma_50'] = analysis.groupby('symbol')['close'].transform(
    lambda x: x.rolling(50).mean()
)
analysis['golden_cross'] = analysis['sma_20'] > analysis['sma_50']
analysis['earnings_yield'] = analysis['eps'] / analysis['close']
analysis['value_score'] = pd.cut(
    analysis['pe'],
    bins=[-float('inf'), 10, 15, 25, float('inf')],
    labels=[2, 1, 0, -1]
).astype(int)
analysis['total_score'] = (
    analysis['value_score'] + analysis['book_score'] +
    analysis['yield_score'] + analysis['momentum_score'] +
    analysis['trend_score']
)
analysis['action'] = analysis['total_score'].apply(
    lambda s: 'BUY' if s >= 5 else 'WATCH' if s >= 2 else 'HOLD' if s >= 0 else 'AVOID'
)

by_action = analysis.groupby('action')['total_score'].mean()
buys = analysis[analysis['action'] == 'BUY'].sort_values('total_score', ascending=False)
```

Namo 1.x

```
ohlcv = Namo.new(ohlcv_data)
fundamentals = Namo.new(fundamentals_data)

analysis = ohlcv.*(fundamentals) do |row, candidates|
 candidates.select{|f| f[:quarter_end] <= row[:date]}.sort_by{|f| f[:quarter_end]}.last(1)
end

analysis[:sma] = proc do |row, namo, field, period|
  window = namo[symbol: row[:symbol], date: ..row[:date]].last(period)
  window.sum{|r| r[field]} / window.count.to_f
end
analysis[:golden_cross] = proc{|row| row[:sma, :close, 20] > row[:sma, :close, 50]}
analysis[:earnings_yield] = proc{|row| row[:eps] / row[:close]}
analysis[:value_score] = proc{|row| (2 if row[:pe] < 10) || (1 if row[:pe] < 15) || (0 if row[:pe] < 25) || -
1}
analysis[:total_score] = proc{|row| row[:value_score] + row[:book_score] + row[:yield_score] + row[:momen
analysis[:action] = proc{|row| row[:total_score] >= 5 ? 'BUY' : row[:total_score] >= 2 ? 'WATCH' : row[:tot

by_action = analysis.group_by(:action).summary(:total_score, reducer: :mean)
buys = analysis[action: 'BUY'].sort_by{|row| -row[:total_score]}
```

Namo 2.x

```
ohlcv = Namo.new(ohlcv_data)
fundamentals = Namo.new(fundamentals_data)

analysis = ohlcv.*(fundamentals) do |row, candidates|
 candidates.select{|f| f[:quarter_end] <= row[:date]}.sort_by{|f| f[:quarter_end]}.last(1)
end

analysis.sma = proc do |row, namo, field, period|
  window = namo[symbol: symbol, date: ..date].last(period)
  window.sum(&field) / window.count.to_f
end
analysis.golden_cross = proc{ sma(:close, 20) > sma(:close, 50) }
analysis.earnings_yield = proc{ eps / close }
analysis.value_score = proc{ (2 if pe < 10) || (1 if pe < 15) || (0 if pe < 25) || -1 }
analysis.total_score = proc{ value_score + book_score + yield_score + momentum_score + trend_score }
analysis.action = proc{ total_score >= 5 ? 'BUY' : total_score >= 2 ? 'WATCH' : total_score >= 0 ? 'HOLD' : '

by_action = analysis.group_by(:action).summary(:total_score, reducer: :mean)
buys = analysis[action: 'BUY'].sort_by{|row| -row.total_score}
```

Namo 3.x

```
ohlcv = Namo.new(ohlcv_data)
fundamentals = Namo.new(fundamentals_data)

analysis = ohlcv.*(fundamentals) do |row, candidates|
 candidates.select{|f| f[:quarter_end] <= row[:date]}.sort_by{|f| f[:quarter_end]}.last(1)
end

analysis = analysis.define do
```

```
  sma          ->(field, period) { namo[symbol: symbol, date: ..date].last(period).sum(&field) / period.to
  golden_cross     -> { sma(:close, 20) > sma(:close, 50) }
  earnings_yield   -> { eps / close }
  value_score      -> { (2 if pe < 10) || (1 if pe < 15) || (0 if pe < 25) || -1 }
  total_score    -> { value_score + book_score + yield_score + momentum_score + trend_score }
  action       -> { total_score >= 5 ? 'BUY' : total_score >= 2 ? 'WATCH' : total_score >= 0 ? 'HOLD' : 'AVOI
end

by_action = analysis.group_by(:action).summary(:total_score, reducer: :mean)
buys = analysis[action: 'BUY'].sort_by{|row| -row.total_score}
```

What to highlight

The progression: Pandas 30+ lines, Namo 1.x ~15 lines, 2.x ~13 lines, 3.x ~12 lines. But the real story isn't line count — it's noise reduction. Each Namo version removes a layer of syntactic ceremony while expressing exactly the same computation.

1.x already wins on composition (`*` with a block) and the parameterised `sma` formula — both shipped behaviour (0.14.0 and 0.17.0). But `row[:close]` is still noisy.

2.x replaces `row[:close]` with `close`. The formulae start reading like mathematical definitions.

3.x removes `analysis.name = proc{ }` and replaces it with `name -> { }`. The define block is a pure specification — just names and computations.

The temporal join in Namo is a block on `*`. In Pandas it's `merge_asof` — a function most users don't know exists.

`group_by(:action)` partitions on a *computed* category — `action` is a formula, yet the call is the same one you'd write for a stored column. Pandas can group by `action` only because it was first materialised as a column; Namo groups by the formula directly, and `summary` returns a `Namo::Collection` you can hold and re-query, not a transient `GroupBy`.

Climate / environmental science

Compose temperature readings with station metadata. Derive anomalies, classify extremes.

xarray

```
readings = xr.open_dataset('temperature.nc')
stations = xr.open_dataset('stations.nc')

climate = xr.merge([readings, stations])

climate['anomaly'] = climate['temperature'] - climate['historical_mean']
climate['classification'] = xr.where(
    climate['anomaly'] > 3.0, 'extreme heat',
    xr.where(climate['anomaly'] < -3.0, 'extreme cold', 'normal')
)
climate['above_treeline'] = climate['elevation'] > 2000
```

Namo 1.x

```
stations = Namo.new(station_data)
readings = Namo.new(temperature_data)
climate = readings * stations
```

```
climate[:anomaly] = proc{|row| row[:temperature] - row[:historical_mean]}
climate[:classification] = proc do |row|
  (row[:anomaly] > 3.0 ? 'extreme heat' :
   row[:anomaly] < -3.0 ? 'extreme cold' :
   'normal')
end
climate[:above_treeline] = proc{|row| row[:elevation] > 2000}
```

Namo 2.x

```
stations = Namo.new(station_data)
readings = Namo.new(temperature_data)
climate = readings * stations

climate.anomaly = proc{ temperature - historical_mean }
climate.classification = proc do
  (anomaly > 3.0 ? 'extreme heat' :
   anomaly < -3.0 ? 'extreme cold' :
   'normal')
end
climate.above_treeline = proc{ elevation > 2000 }
```

Namo 3.x

```
stations = Namo.new(station_data)
readings = Namo.new(temperature_data)

climate = (readings * stations).define do
  anomaly          -> { temperature - historical_mean }
  classification  -> { (anomaly > 3.0 ? 'extreme heat' : anomaly < -3.0 ? 'extreme cold' : 'normal') }
  above_treeline   -> { elevation > 2000 }
end
```

What to highlight

xarray requires `xr.where` nesting for conditional classification — each level adds indentation. Namo uses Ruby's ternary at every stage.

The 3.x version compresses the entire analysis to six lines including data loading. The composition and definition happen in a single chained expression.

`readings * stations` is the hook — one operator, automatic dimension matching on `station`. A climate scientist reading this sees the join as natural, not as plumbing.

Genomics / bioinformatics

Gene expression data composed with sample metadata. Identify differentially expressed genes.

R (dplyr)

```
study <- expression %>%
  inner_join(metadata, by = 'sample') %>%
  mutate(
    fold_change = treated_mean / control_mean,
    significant = p_value < 0.05 & abs(fold_change) > 2.0,
```

```
    category = case_when(
      fold_change > 2.0 ~ 'upregulated',
      fold_change < -2.0 ~ 'downregulated',
      TRUE ~ 'unchanged'
    )
  )

upregulated <- study %>%
  filter(significant == TRUE, category == 'upregulated')
```

Namo 1.x

```
expression = Namo.new(expression_data)
metadata = Namo.new(sample_metadata)
study = expression * metadata

study[:fold_change] = proc{|row| row[:treated_mean] / row[:control_mean]}
study[:significant] = proc{|row| row[:p_value] < 0.05 && row[:fold_change].abs > 2.0}
study[:category] = proc do |row|
  (row[:fold_change] > 2.0 ? 'upregulated' :
   row[:fold_change] < -2.0 ? 'downregulated' :
   'unchanged')
end

upregulated = study[significant: true, category: 'upregulated']
```

Namo 2.x

```
expression = Namo.new(expression_data)
metadata = Namo.new(sample_metadata)
study = expression * metadata

study.fold_change = proc{ treated_mean / control_mean }
study.significant = proc{ p_value < 0.05 && fold_change.abs > 2.0 }
study.category = proc do
  (fold_change > 2.0 ? 'upregulated' :
   fold_change < -2.0 ? 'downregulated' :
   'unchanged')
end

upregulated = study[significant: true, category: 'upregulated']
```

Namo 3.x

```
expression = Namo.new(expression_data)
metadata = Namo.new(sample_metadata)

study = (expression * metadata).define do
  fold_change   -> { treated_mean / control_mean }
  significant   -> { p_value < 0.05 && fold_change.abs > 2.0 }
    category           -> { (fold_change > 2.0 ? 'upregulated' : fold_change < -
2.0 ? 'downregulated' : 'unchanged') }
end
```

```
upregulated = study[significant: true, category: 'upregulated']
```

What to highlight

R's dplyr is the strongest competitor here — the pipe operator and `case_when` are clean. Acknowledge this.

The selection line is where Namo wins at every version: `study[significant: true, category: 'upregulated']` reads like English. Even R's `filter(significant == TRUE, category == 'upregulated')` has more syntactic weight.

In the 3.x define block, the entire differential expression analysis is three formula lines plus one selection line. A bioinformatician reads the biology, not the tooling.

Epidemiology / public health

Daily case counts per region. Derive a rolling weekly average and a trend from it — cross-row computation, where each row's value reads the rows around it.

Polars

```
cases = cases.sort('date')

cases = cases.with_columns(
    pl.col('new_cases').rolling_mean(window_size=7, min_samples=1)
      .over('region').alias('weekly_average')
)
cases = cases.with_columns(
    pl.when(pl.col('new_cases') > pl.col('weekly_average'))
      .then(pl.lit('rising')).otherwise(pl.lit('falling')).alias('trend')
)

by_region = cases.group_by('region').agg(pl.col('new_cases').sum())
rising = cases.filter(pl.col('trend') == 'rising')
```

Namo 1.x

```
cases = Namo.new(case_data)

cases[:weekly_average] = proc do |row, namo|
  window = namo[region: row[:region], date: ..row[:date]].last(7)
  window.values(:new_cases).sum / window.count.to_f
end
cases[:trend] = proc{|row| row[:new_cases] > row[:weekly_average] ? 'rising' : 'falling'}

by_region = cases.group_by(:region).summary(:new_cases, reducer: :sum)
rising = cases[trend: 'rising']
```

Namo 2.x

```
cases = Namo.new(case_data)

cases.weekly_average = proc do |row, namo|
  window = namo[region: region, date: ..date].last(7)
  window.sum(&:new_cases) / window.count.to_f
end
```

```
cases.trend = proc{ new_cases > weekly_average ? 'rising' : 'falling' }

by_region = cases.group_by(:region).summary(:new_cases, reducer: :sum)
rising = cases[trend: 'rising']
```

Namo 3.x

```
cases = Namo.new(case_data).define do
  weekly_average -> {
    window = namo[region: region, date: ..date].last(7)
    window.sum(&:new_cases) / window.count.to_f
  }
  trend          -> { new_cases > weekly_average ? 'rising' : 'falling' }
end

by_region = cases.group_by(:region).summary(:new_cases, reducer: :sum)
rising = cases[trend: 'rising']
```

What to highlight

The two-arity proc is the centre of this example, and it is shipped behaviour (0.15.0): a formula whose parameters are `(row, namo)` receives the Namo the row belongs to, so the window is an ordinary selection — the same `[]` interface used everywhere else, narrowed to the row's region and to dates up to the row's own. There is no separate windowing API to learn.

Polars needs the data sorted first, and the window is assembled from three concepts — `rolling_mean` for the computation, `over` for the partition, `min_samples` for the leading edge. In Namo the partition is `region: row[:region]`, the bound is the range `date: ..row[:date]`, and the leading edge needs nothing: `last(7)` of a three-row window is the three rows.

The window is live. Append today's counts and `weekly_average` reads the new rows on next access; filter the Namo and the filtered result's rows window over the filtered rows. In Polars the rolling column is a snapshot — new data means recomputing it.

`trend` references `weekly_average` — a one-arity formula reading a two-arity one by name. The two forms mix freely.

`group_by(:region)` partitions on a stored dimension — the same call as the computed-category groupings elsewhere, here over plain data. Because `region` is a data dimension, the split is lossless and exactly reversible: `cases.group_by(:region).as_detail(:region) == cases`. The partition reorganises the rows; it doesn't lose them.

Survey / social science

Survey responses composed with demographics. Analyse response patterns.

Polars

```
study = responses.join(demographics, on='respondent')

study = study.with_columns([
    pl.when(pl.col('response') >= 4).then(pl.lit('satisfied'))
      .when(pl.col('response') <= 2).then(pl.lit('dissatisfied'))
      .otherwise(pl.lit('neutral')).alias('satisfaction'),
])
study = study.with_columns([
```

```
    (pl.col('age_group') + pl.lit(', ') + pl.col('region') +
     pl.lit(': ') + pl.col('satisfaction')).alias('label'),
])

by_satisfaction = study.group_by('satisfaction').agg(pl.len())
dissatisfied_young = study.filter(
    (pl.col('satisfaction') == 'dissatisfied') &
    (pl.col('age_group') == 'under_30')
)
```

Namo 1.x

```
responses = Namo.new(survey_data)
demographics = Namo.new(demographic_data)
study = responses * demographics

study[:satisfaction] = proc{|row| row[:response] >= 4 ? 'satisfied' : row[:response] <= 2 ? 'dissatisfied'
study[:label] = proc{|row| "#{row[:age_group]}, #{row[:region]}: #{row[:satisfaction]}"}

by_satisfaction = study.group_by(:satisfaction).summary(:respondent, reducer: :count)
dissatisfied_young = study[satisfaction: 'dissatisfied', age_group: 'under_30']
```

Namo 2.x

```
responses = Namo.new(survey_data)
demographics = Namo.new(demographic_data)
study = responses * demographics

study.satisfaction = proc{ response >= 4 ? 'satisfied' : response <= 2 ? 'dissatisfied' : 'neutral' }
study.label = proc{ "#{age_group}, #{region}: #{satisfaction}" }

by_satisfaction = study.group_by(:satisfaction).summary(:respondent, reducer: :count)
dissatisfied_young = study[satisfaction: 'dissatisfied', age_group: 'under_30']
```

Namo 3.x

```
responses = Namo.new(survey_data)
demographics = Namo.new(demographic_data)

study = (responses * demographics).define do
  satisfaction  -> { response >= 4 ? 'satisfied' : response <= 2 ? 'dissatisfied' : 'neutral' }
   label         -> { "#{age_group}, #{region}: #{satisfaction}" }
end

by_satisfaction = study.group_by(:satisfaction).summary(:respondent, reducer: :count)
dissatisfied_young = study[satisfaction: 'dissatisfied', age_group: 'under_30']
```

What to highlight

Polars is Python's best modern option, and it struggles here. The `pl.when().then().when().then().otherwise()` chain for a simple three-way classification is five method calls. Namo's ternary is one expression at every version.

String concatenation in Polars requires `pl.col() + pl.lit()` chaining — building a string from column

references and literal connectors. Ruby's string interpolation (`"#{age_group}, #{region}: #{satisfaction}"`) does the same thing naturally, and in 2.x/3.x the interpolated names are bare.

The `label` formula in Namo references `satisfaction` — another computed formula — inside the interpolation. In Polars, `satisfaction` must exist as a column from a previous `with_columns` call before it can be referenced. Order matters. In Namo, lazy resolution means it doesn't.

Type agnosticism is the key point for social scientists. Every formula here produces text, not numbers. The tool doesn't force numeric thinking.

`group_by(:satisfaction)` is the response-pattern breakdown the discipline is named for. `satisfaction` is computed text, yet it partitions exactly like a stored column — the count per level falls straight out of `summary`. Polars groups too, but the result is a transient frame; Namo's is a `Namo::Collection`, kept and re-queried alongside the row-level `study`.

Sports analytics

Three-way composition: player stats, team data, match data.

Pandas

```
analysis = stats.merge(teams, on='player').merge(matches, on='match')

analysis['goals_per_90'] = (analysis['goals'] / analysis['minutes']) * 90
analysis['contribution'] = analysis['goals'] + analysis['assists']
analysis['home_advantage'] = analysis['venue'].apply(
    lambda v: 'home' if v == 'home' else 'away'
)
analysis['form'] = analysis.groupby('player')['goals_per_90'].transform(
    lambda x: x.rolling(5).mean()
)

top_scorers = (
    analysis[analysis['position'] == 'forward']
    .sort_values('goals_per_90', ascending=False)
)
```

Namo 1.x

```
stats = Namo.new(player_stats)
teams = Namo.new(team_data)
matches = Namo.new(match_data)
analysis = stats * teams * matches

analysis[:goals_per_90] = proc{|row| (row[:goals] / row[:minutes].to_f) * 90}
analysis[:contribution] = proc{|row| row[:goals] + row[:assists]}
analysis[:home_advantage] = proc{|row| row[:venue] == 'home' ? 'home' : 'away'}
analysis[:form] = proc{|row, namo| row[:sma, :goals_per_90, 5]}

top_scorers = analysis[position: 'forward'].sort_by{|row| -row[:goals_per_90]}
```

Namo 2.x

```
stats = Namo.new(player_stats)
teams = Namo.new(team_data)
matches = Namo.new(match_data)
```

```
analysis = stats * teams * matches

analysis.goals_per_90 = proc{ (goals / minutes.to_f) * 90 }
analysis.contribution = proc{ goals + assists }
analysis.home_advantage = proc{ venue == 'home' ? 'home' : 'away' }
analysis.form = proc{ sma(:goals_per_90, 5) }

top_scorers = analysis[position: 'forward'].sort_by{|row| -row.goals_per_90}
```

Namo 3.x

```
stats = Namo.new(player_stats)
teams = Namo.new(team_data)
matches = Namo.new(match_data)

analysis = (stats * teams * matches).define do
  goals_per_90     -> { (goals / minutes.to_f) * 90 }
  contribution     -> { goals + assists }
  home_advantage   -> { venue == 'home' ? 'home' : 'away' }
  form             -> { sma(:goals_per_90, 5) }
end

top_scorers = analysis[position: 'forward'].sort_by{|row| -row.goals_per_90}
```

What to highlight

`stats * teams * matches` — three data sources composed with two operators. Pandas needs two chained `.merge()` calls, each specifying join keys. This is the hook for any audience.

The `form` formula reads as "the 5-period moving average of goals per 90." In Pandas, it's `groupby` + `transform` + `rolling` — three concepts chained together, referencing a column by string name.

The 3.x define block makes the analytical model self-documenting — four metrics, each one line, each readable without knowing the tooling.

Supply chain / logistics

Shipment data composed with route costs. Identify unprofitable routes.

Julia (DataFrames.jl)

```
logistics = innerjoin(shipments, routes, on = [:origin, :destination])

logistics.shipping_cost = logistics.weight .* logistics.cost_per_kg
logistics.margin = logistics.revenue .- logistics.shipping_cost
logistics.profitable = logistics.margin .> 0

unprofitable_routes = select(
    filter(:profitable => !, logistics),
    Not([:shipment_id, :weight, :date])
)
```

Namo 1.x

```
shipments = Namo.new(shipment_data)
```

```
routes = Namo.new(route_costs)
logistics = shipments * routes

logistics[:shipping_cost] = proc{|row| row[:weight] * row[:cost_per_kg]}
logistics[:margin] = proc{|row| row[:revenue] - row[:shipping_cost]}
logistics[:profitable] = proc{|row| row[:margin] > 0}

unprofitable_routes = logistics[profitable: false][-:shipment_id, -:weight, -:date]
```

Namo 2.x

```
shipments = Namo.new(shipment_data)
routes = Namo.new(route_costs)
logistics = shipments * routes

logistics.shipping_cost = proc{ weight * cost_per_kg }
logistics.margin = proc{ revenue - shipping_cost }
logistics.profitable = proc{ margin > 0 }

unprofitable_routes = logistics[profitable: false][-:shipment_id, -:weight, -:date]
```

Namo 3.x

```
shipments = Namo.new(shipment_data)
routes = Namo.new(route_costs)

logistics = (shipments * routes).define do
  shipping_cost -> { weight * cost_per_kg }
  margin        -> { revenue - shipping_cost }
  profitable    -> { margin > 0 }
end

unprofitable_routes = logistics[profitable: false][-:shipment_id, -:weight, -:date]
```

What to highlight

Julia is fast and its syntax is clean, but the broadcasting operators (`.*`, `.-`, `.>`) are visual noise — every arithmetic operation needs a dot prefix for element-wise computation. Namo's formulae are plain arithmetic because they operate at the row level.

Julia's column removal uses `Not([:shipment_id, :weight, :date])` — a negation wrapper around an array of symbols. Namo's `-:shipment_id, -:weight, -:date` reads as natural subtraction of dimensions. The contraction syntax is unique to Namo and visually lighter.

The last line is identical across all Namo versions: `logistics[profitable: false][-:shipment_id, -:weight, -:date]`. Selection on a computed boolean, then contraction to focus the output.

Julia's `filter(:profitable => !, logistics)` passes a negation function — clever but cryptic. Namo's `profitable: false` is keyword selection with a value.

Astronomy / physics

Stellar observations composed with catalogue data. Derive physical properties, classify, select.

R

```
survey <- observations %>%
  inner_join(catalogue, by = 'star_id') %>%
  mutate(
    absolute_magnitude = magnitude - 5 * log10(distance_ly / 10.0),
    luminosity_class = case_when(
      absolute_magnitude < -5 ~ 'supergiant',
      absolute_magnitude < 0 ~ 'giant',
      absolute_magnitude < 5 ~ 'main sequence',
      TRUE ~ 'dwarf'
    )
  )

nearby_giants <- survey %>%
  filter(distance_ly < 500, luminosity_class == 'giant')
```

Namo 1.x

```
observations = Namo.new(observation_data)
catalogue = Namo.new(star_catalogue)
survey = observations * catalogue

survey[:absolute_magnitude] = proc{|row| row[:magnitude] - 5 * Math.log10(row[:distance_ly] / 10.0)}
survey[:luminosity_class] = proc do |row|
  (row[:absolute_magnitude] < -5 ? 'supergiant' :
   row[:absolute_magnitude] < 0 ? 'giant' :
   row[:absolute_magnitude] < 5 ? 'main sequence' :
   'dwarf')
end

nearby_giants = survey[
  distance_ly: ->(v){ v < 500 },
  luminosity_class: 'giant'
]
```

Namo 2.x

```
observations = Namo.new(observation_data)
catalogue = Namo.new(star_catalogue)
survey = observations * catalogue

survey.absolute_magnitude = proc{ magnitude - 5 * Math.log10(distance_ly / 10.0) }
survey.luminosity_class = proc do
  (absolute_magnitude < -5 ? 'supergiant' :
   absolute_magnitude < 0 ? 'giant' :
   absolute_magnitude < 5 ? 'main sequence' :
   'dwarf')
end

nearby_giants = survey[
  distance_ly: ->(v){ v < 500 },
  luminosity_class: 'giant'
]
```

Namo 3.x

```
observations = Namo.new(observation_data)
catalogue = Namo.new(star_catalogue)

survey = (observations * catalogue).define do
  absolute_magnitude -> { magnitude - 5 * Math.log10(distance_ly / 10.0) }
 luminosity_class  -> { (absolute_magnitude < -5 ? 'supergiant' : absolute_magnitude < 0 ? 'giant' : absolu
end

nearby_giants = survey[
  distance_ly: ->(v){ v < 500 },
  luminosity_class: 'giant'
]
```

What to highlight

R is strong here — `case_when` is readable and the pipe operator chains cleanly. Acknowledge this.

`Math.log10` inside a formula — Ruby's full standard library is available inside procs. R has built-in `log10` which is similarly clean, so this is a wash.

The proc-based selection `distance_ly: ->(v){ v < 500 }` is unique to Namo. It passes a lambda as a selection predicate — a range-like query without range syntax. R's `filter(distance_ly < 500)` is clean but it's a function call, not a data access pattern. Namo unifies selection by value, by range, and by predicate in a single `[]` interface.

The 3.x define block reduces the physics to two formula lines. An astronomer sees the distance modulus and the HR diagram classification — the physics, not the tooling.

Presentation design principles

When presenting these examples:

- Show all four stages in sequence: current tool, 1.x, 2.x, 3.x. The progression is the argument. Each step removes ceremony while preserving meaning.
- Pause on 1.x. It's already better than the comparison tool — `*` composition, keyword selection, proc formulae. Let the audience absorb that before showing bare names.
- The 1.x to 2.x transition is the biggest visual change — `row[:close]` becomes `close`. Give it a moment.
- The 2.x to 3.x transition is subtler — `analysis.name = proc{ }` becomes `name -> { }`. The DSL block is polish, not transformation. Present it as "and this is where it's going" rather than "this is the big reveal."
- Count the repetitions. In Pandas, `analysis['column']` repeats the variable name endlessly. In Polars, `pl.col('column')` does the same. In Namo 1.x, `row[:name]` repeats `row`. In 2.x, bare names eliminate even that. The visual noise reduction is striking when shown side by side.
- Acknowledge where competitors are good. R's dplyr is clean for genomics. Julia is fast and expressive. Being honest about competitors' strengths makes the points where Namo wins more credible.
- The selection line is the closer for every example. `study[significant: true, category: 'upregulated']` on computed textual dimensions is the thing no other tool does concisely. Save it for last.
- For mixed audiences, show all seven disciplines briefly, then deep-dive into the one most relevant to the room. Breadth first, then depth.

- Have a live IRB session ready. Type the examples live. The immediacy of the response — data in, result out, no boilerplate — is more convincing than slides.